004 MockLLM

2026年9月3日 13:21

MockLLM 是指“模拟大语言模型”（Mock Large Language Model），在开发 AI 应用（尤其是 AI Harness、编码助手）时，用于模拟真实模型行为的测试替身。它不调用真实 API，而是返回预定义的响应或根据简单规则生成输出。

1. 为什么需要 MockLLM?

- 成本控制：调用真实 LLM API 需要付费，测试时大量调用成本高昂。
- 速度与稳定性：本地模拟响应极快，且不受网络、限流、服务波动影响。
- 可重复性：固定输入得到固定输出，便于编写单元测试和回归测试。
- 隔离测试：在测试上下文管理、工具调用、Agent 循环等逻辑时，不需要真实模型的“智能”，只需验证框架是否正确处理模型的输出格式（如工具调用请求、流式 token）。
- 模拟边界情况：可以模拟超长输出、错误格式、超时、工具调用异常等，验证 harness 的鲁棒性。

2. MockLLM 在 Coder 类 Harness 中的典型用途

- 测试上下文组装：验证在给定代码库状态下，harness 是否正确选择了要发送给模型的消息列表（如是否包含了预期的代码片段、摘要、工具结果）。
- 测试工具调用循环：模拟模型返回一个 tool_calls 请求，检查 harness 是否能正确执行工具并将结果反馈给模型，继续生成。
- 测试上下文压缩：模拟超长对话，触发压缩策略，验证压缩后上下文仍包含关键信息（如任务目标）。
- 测试流式输出：模拟分块返回 token，确保 UI 能正确处理。
- 开发阶段快速原型：在没有可用 API 密钥或离线环境下，也

能开发和调试整个应用流程。

3. 如何实现一个简单的 MockLLM？

通常实现一个与真实模型接口一致的类，例如在 Python 中：

```
class MockLLM:
    def __init__(self, responses=None):
        # responses 可以是队列或根据输入返回的规则
        self.responses = responses or []
        self.calls = []  # 记录调用历史
def chat(self, messages, tools=None):
        self.calls.append({"messages": messages,
"tools": tools})
        if self.responses:
            return self.responses.pop(0)
        # 默认返回一个简单回复
        return {"role": "assistant", "content": "Mock
response"}
```

在 C++/Qt 环境中，可以创建一个 IModelPlugin 的 Mock 实现，注册到插件系统，用于测试。

4. MockLLM 与上下文管理的关系

上下文压缩、增量组织、工具调用等，都可以通过 MockLLM 进行白盒测试：

- 检查每次调用时传入的 messages 列表是否符合预期（比如是否包含摘要、是否丢弃了旧消息）。
- 通过模拟模型返回工具调用，验证工具执行后上下文是否正确更新。
- 模拟模型返回超长输出，观察上下文窗口是否触发压缩逻辑。

5. 现成的 MockLLM 工具/库

- Python：unittest.mock 可以轻松创建 mock 对象；

LangChain 提供 FakeListLLM 等测试类。
- JavaScript/TypeScript：LangChain.js 也有 FakeListChatModel。
- 通用：自己实现一个简单的类，继承或实现模型接口。